ensive Study of ibuted Systems

re, Consensus, and Fault Tolerance in Modern Infrastructure

Max Mustermann

Technische Universität Berlin
Fakultät für Elektrotechnik und Informatik

May 2026

Contents

.

CHAPTER 1

Foundations of Distributed Systems

Introduction

Distributed systems form the backbone of modern computing infrastructure. From cloud services to social networks, the ability to coordinate multiple autonomous machines is fundamental to scalability and reliability.

1.1 Motivation and Goals

The primary motivation for studying distributed systems lies in their ubiquity. Every time you use a web application, stream a video, or check your email, you are interacting with a distributed system.

- **Why Distributed?**

Centralized systems, while simpler to reason about, face inherent limitations in scale, fault tolerance, and geographic distribution. A single point of failure can bring down an entire service.

1.2 Key Concepts

The following concepts are essential for understanding distributed systems:

Consensus Agreement among distributed nodes on a single data value or system state, despite failures and network delays.

Quorum The minimum number of nodes required to agree before an operation is considered successful, typically a majority.

Partition Tolerance The ability of a system to continue operating despite network partitions that isolate subsets of nodes.

CHAPTER 2

Consensus Algorithms

Algorithm Comparison

Various consensus algorithms offer different trade-offs between safety guarantees, fault tolerance, and implementation complexity.

Table 1. Comparison of Consensus Algorithms

Algorithm	Consensus	Tolerance	Complexity
Paxos	Safety	$f < n/2$	Moderate
Raft	Safety	$f < n/2$	Low
Viewstamped	Safety	$f < n/2$	Moderate
PBFT	Safety	$f < n/3$	High
Zab	Total	$f < n/2$	Moderate
Tendermint	Total	$f < n/3$	Moderate
HotStuff	Total	$f < n/3$	Low

Raft Protocol

Raft was designed as an understandable alternative to Paxos, decomposing the consensus problem into independent subproblems: leader election, log replication, and safety.

3.1 Leader Election Process

The leader election follows a clear sequence of steps:

1. Follower does not receive heartbeat within election timeout
2. Follower increments its term and becomes a candidate
   a. Candidate votes for itself
   b. Candidate sends RequestVote RPCs to all peers
3. Candidate waits for votes from majority of nodes
4. If majority grants votes, candidate becomes leader
5. Leader sends periodic heartbeats to maintain authority

3.2 Log Replication

Once a leader is established, clients send commands to the leader, which appends them to its log and replicates them across followers using AppendEntries RPCs.

- Leader appends command to its local log
  - Each entry stores command and term number
  - Entry index monotonically increases
- Leader sends AppendEntries to all followers
- Followers append entry if log is consistent
- Leader commits once majority acknowledges

CHAPTER 3

Implementation Details

Code Example

A minimal Raft-style consensus node implementation in Python:

```python
class RaftNode:
    def __init__(self, node_id: int, peers: list):
        self.id = node_id
        self.peers = peers
        self.state = "follower"
        self.term = 0
        self.log = []
        self.voted_for = None

    def request_vote(self, candidate_term: int):
        """Handle incoming RequestVote RPC."""
        if candidate_term > self.term:
            self.term = candidate_term
            self.voted_for = None
            self.state = "follower"
        return self._grant_vote(candidate_term)
```

Listing 1. Raft node skeleton

Key Insight

> *“ A distributed system is one in which the failure of a computer you didn’t even know existed can render your own computer unusable. — Leslie Lamport ”*

This fundamental observation underscores the importance of fault tolerance as a first-class design concern in distributed systems.

```ini
[cluster]
node_id = "node-1"
bind_addr = "0.0.0.0:7946"
election_timeout = "1.5s"
heartbeat_timeout = "500ms"
```

Listing 2. Minimal configuration